AES

Elisabeth Oswald

ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

AES — Implementations — Secret Sharing

Our first “big achievement” in this module will be to develop an understanding of how secret sharing is used in embedded systems security to secure implementations of encryption schemes, particularly, AES.

Some embedded systems are seen as “critical”: integrated circuits (ICs) and their firmware in applications like banking (smart cards, but also phones), ID cards, passports, but also in critical infrastructures, etc.

They can only be sold if they can withstand security evaluations (of certified evaluators) with regards to a “high attack potential”: this means that they have to be secure against side channel (and other) attacks using millions of side channel observations.

This is achieved via “masking”, which is another jargon term for secret sharing.

AES - Input, Output, Key

AES operates exclusively on message blocks that have 128 bits or 16 bytes $m_0, \ldots, m_{15}$.

AES keys can be either 128, 192, or 256 bits. In this module we only consider 128 bit keys. This does not impact on how secret sharing is implemented.

AES takes as input a message block and a key, and both are organised in a 4 by 4 matrix.

m_0	m_4	m_8	m_{12}
m_1	m_5	m_9	m_{13}
m_2	m_6	m_{10}	m_{14}
m_3	m_7	m_{11}	m_{15}

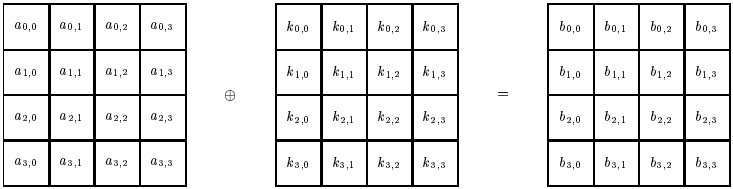
k_0	k_4	k_8	k_{12}
k_1	k_5	k_9	k_{13}
k_2	k_6	k_{10}	k_{14}
k_3	k_7	k_{11}	k_{15}

Figure: State and Key Layout (numbering convention) for AES

AES

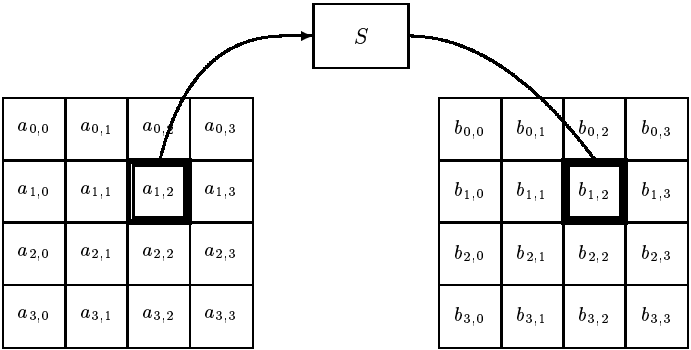
```
1  AES_Encrypt(ByteArray m, ByteArray k)
2   ByteArray c
3   ByteArray s = m
4   rk = KeyExpansion(k)
5   AddRoundKey(s, rk^0)
6   for r = 1 step 1 to 9
7               SubBytes(s)
8               ShiftRows(s)
9               MixColumns(s)
10              AddRoundKey(s, rk^r)
11   end
12   SubBytes(s)
13   ShiftRows(s)
14   AddRoundKey(s, rk^10)
15  return c = s;
```

AES - AddRoundKey



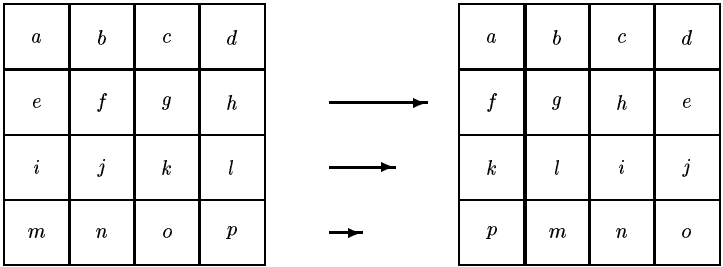
AddRoundKey is a binary addition of state bytes, which can be accomplished via the Boolean exclusive-or.

AES - Byte Substitution



State matrix is worked through byte by byte.
S-box is an algebraic function that is based on a finite field inversion. A single S-box function is applied to all state bytes. The finite field inversion is a non-linear function, and it does not commute with a Boolean exclusive-or.

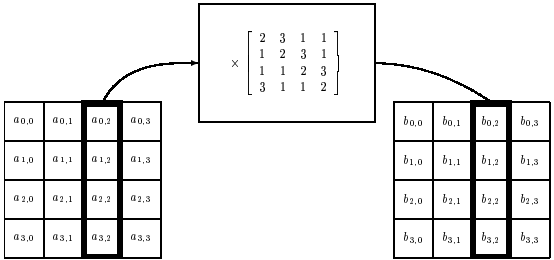
AES - Shift Rows



Shift Rows actually left rotates the rows of the state matrix.

It makes sense when you consider it together with the action of MixColumns.

AES - Mix Columns



- This is defined as a polynomial multiplication over the finite field $\mathbb{Z}_{2^8}$.
- It mixees up bytes within a column of the state.

ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

Extension Fields

The AES state bytes are bit strings: we know already that bits and operations on bits, can be understood as arithmetic in $\mathbb{Z}_2$ (the finite field with two elements, in which all operations are done $\pmod 2$).

We can define **polynomials** over $\mathbb{Z}_2$ (i.e. polynomials whose coefficients are elements of $\mathbb{Z}_2$): $f(X) = \sum_i a_i X^i \pmod 2$, e.g. $1 + X^2 + X^4$, or $1 + X$.

You can use "bit strings" as a shorthand for these polynomials. Suppose that the "largest" polynomial that we want to represent has 8 coefficients, then the polynomials from before would be $(1, 0, 1, 0, 1, 0, 0, 0)$ and $(1, 1, 0, 0, 0, 0, 0, 0)$.

Consider the addition of two such polynomials: $(1 + X^2 + X^4) + (1 + X) = X + X^2 + X^4$.

Consider the multiplication of two such polynomials:
$(1 + X^2 + X^4) \cdot (1 + X) = 1 + X + X^2 + X^3 + X^4 + X^5$.

Over to You

Convert the following polynomials into a number in base 2, in base 10, as well as base 16:

$$X + X^2 + X^4$$

$$1 + X + X^2 + X^3 + X^4 + X^5$$

Extension Fields

The arithmetic looks more complicated but:

$$f(x) \equiv r(x) \pmod{p(x)}$$

because there is an $a(x)$ such that $f(x) = a(x) \cdot p(x) + r(x)$ (and all polynomials have coefficients in some $\mathbb{Z}_p$).

Like with arithmetic in $\mathbb{Z}_p$, we get guaranteed invertibility if we choose a "special" polynomial $p(x)$.

This will be a special polynomial, that has as only factors 1 and itself (so the equivalent of a prime modulus). These polynomials are called **irreducible**.

With these we can define an extension to any finite field $\mathbb{Z}_p$ as $(\mathbb{Z}_p[x]/p(x)\mathbb{Z}_p[x])$: like with ordinary numbers we can define "residue classes" (i.e. remainders after polynomial division), in which we can "sort" all polynomials (with coefficients over $\mathbb{Z}_p$) that have a degree that is larger than the degree of the irreducible polynomial.

I call the irreducible polynomial now $p(x)$.

It specifies the "wrap around": $p(x) \equiv 0 \pmod{p(x)}$.
(Remember that we doing things $\pmod{p}$, then $p \equiv 0 \pmod{p}$, $p+1 \equiv 1 \pmod{p}$, etc.)

So if $p(x) = x^d + p'(x)$, then $x^d \equiv p'(x)$.

In the example before we had $p(x) = x^4 + 1$, so $x^4 + 1 \equiv 0$, and $x^4 \equiv 1 \pmod{p(x)}$ (which is the trick that I used in the examples). So $x^5 = x \cdot x^4 \equiv x$, or $x^4 + x^5 = x^4(x+1) = x + 1$.

Extension Fields

Addition of two polynomials will give another polynomial of the same degree.
Multiplication of two polynomials increases the degree of the result.

If we want to stay within a finite number of polynomials (=elements) we need something like a "modulus".

As we are working with polynomials now, we choose a polynomial of a certain degree as a module. E.g. if we want to restrict the degree of all polynomials to be at most d, then we choose a polynomial of degree $d+1$.

E.g. take X^4+1 to be our modulus, so $X^4+1 \equiv 0 \pmod{X^4+1}$:
$(1+X^2+X^4)+(1+X) \equiv (X^2)+(1+X) = 1+X+X^2 \pmod{1+X^4}$.

$(1+X^2+X^4)\cdot(1+X) \equiv X^2\cdot(1+X) = X^2+X^3 \pmod{1+X^4}$.

Over to You

Calculate, with paper and pencil (or the electronic equivalent), the following:

$(x^3 + x + 1) \cdot (x + 1) \pmod{x^4 + 1}$, with coefficients in $\mathbb{Z}_2$

$(x^2 + 2x + 1) \cdot 2x \pmod{x^3 + 1}$, with coefficients in $\mathbb{Z}_3$

AES Extension Field

Based on bits, i.e. elements of $\mathbb{Z}_2$, we can define bytes as elements of an extension field of $\mathbb{Z}_2$, via finding/specifying an irreducible polynomial of degree 8.

The choice of the irreducible polynomial is important, as different polynomials impact on the efficiency of multiplication and inversion.

In AES the choice of the irreducible polynomial is $p(x) = x^8 + x^4 + x^3 + x + 1$.

Hence the finite field extension in AES for state bytes is $(\mathbb{Z}_2[x]/p(x)\mathbb{Z}_2[x])$: it is commonly abbreviated as $\mathbb{Z}_{2^8}$.

Modular multiplication in MixColumns

MixColumns takes a column of the AES state, and interprets it as a further extension over $\mathbb{Z}_{2^8}$:

$$s_i(X) = s_{3,i}X^3 + s_{2,i}X^2 + s_{1,i}X + s_{0,i}$$

(i-th column), with $s_{j,i} \in \mathbb{Z}_{2^8}$.

Thus a column is also understood as a polynomial.

Hence this corresponds to a further extension of degree 4 over $\mathbb{Z}_{2^8}$. This extension is done via the **reducible** polynomial $X^4 + 1$.

So the column multiplication in MixColumns is a multiplication of polynomials, but now the polynomials have coefficients over $\mathbb{Z}_{2^8}$.

Modular multiplication in MixColumns, cont.

The column multiplication takes a column-polynomial and multiplies it with the polynomial $a(X) = 3X^3 + X^2 + X + 2$. The top-level multiplication is done modulo $X^4 + 1$. Each element-wise multiplication is done modulo $p(x)$.

!!Warning!!: the multiplication of the column is in the extension over $\mathbb{Z}_{2^8}$ but the multiplications $s_{i,j} \cdot a_i$ is in $\mathbb{Z}_{2^8}$.

This is why I write the variable in capital letters, so X, to indicate that we have moved to an ever larger algebraic structure.

The column multiplication can be conveniently expressed as a kind of matrix multiplication.

Over to You

Work out the result of

$$(X^3 + X^2 + X + 1) \cdot (3X^3 + X^2 + X + 2) \in (\mathbb{Z}_{2^8}[X]/(X^4 + 1)\mathbb{Z}_{2^8}[X]).$$

(Beware that the coefficients here are elements in $\mathbb{Z}_{2^8}$, so they must be understood as polynomials over $\mathbb{Z}_2$.)

Matrix multiplication in MixColumns

For polynomials $a(X), s_i(X)$ over $\mathbb{Z}_{2^8}$, the multiplication $\pmod{X^4+1}$, is given as:

$$\begin{aligned} a(X) \cdot s_i(X) = c(X) &= \left(a_3X^3 + a_2X^2 + a_1X + a_0\right) \cdot \left(s_{3,i}X^3 + s_{2,i}X^2 + s_{1,i}X + s_{0,i}\right) \\ &= c_6X^6 + c_5X^5 + c_4X^4 + c_3X^3 + c_2X^2 + c_1X + c_0 \\ &\equiv d_3X^3 + d_2X^2 + d_1X + d_0 \end{aligned}$$

Evidently the coefficients d_i are products of the coefficients of $a(X) = 3X^3 + X^2 + X + 2$ and $s(X)$.

$$\begin{aligned} d_0 &= a_0 \cdot s_{0,i} + a_3 \cdot s_{1,i} + a_2 \cdot s_{2,i} + a_1 \cdot s_{3,i} = 2 \cdot s_{0,i} + 3 \cdot s_{1,i} + 1 \cdot s_{2,i} + 1 \cdot s_{3,i} \\ d_1 &= a_1 \cdot s_{0,i} + a_0 \cdot s_{1,i} + a_3 \cdot s_{2,i} + a_2 \cdot s_{3,i} = 1 \cdot s_{0,i} + 2 \cdot s_{1,i} + 3 \cdot s_{2,i} + 1 \cdot s_{3,i} \\ d_2 &= a_2 \cdot s_{0,i} + a_1 \cdot s_{1,i} + a_0 \cdot s_{2,i} + a_3 \cdot s_{3,i} = 1 \cdot s_{0,i} + 1 \cdot s_{1,i} + 2 \cdot s_{2,i} + 3 \cdot s_{3,i} \\ d_3 &= a_3 \cdot s_{0,i} + a_2 \cdot s_{1,i} + a_1 \cdot s_{2,i} + a_0 \cdot s_{3,i} = 3 \cdot s_{0,i} + 1 \cdot s_{1,i} + 1 \cdot s_{2,i} + 2 \cdot s_{3,i} \end{aligned}$$

Matrix multiplication in MixColumns

We still need to consider how to calculate $a_i \cdot s_{j,i} \pmod{x^8 + x^4 + x^3 + x + 1}$. The coefficients a_i, although represented as decimal numbers before, are of course also elements of $\mathbb{Z}_{2^8}$.

So $a_3 = 3$ is actually the polynomial $x + 1$, $a_2 = a_1 = 1$, and $a_0 = 2 = x$.

Therefore we need to perform multiplication by x over $\mathbb{Z}_{2^8}$. This operation is often referred as `xtime`: `xtime`=$x \cdot f(x) \pmod{x^8 + x^4 + x^3 + x + 1}$.

$f(x)$ has degree smaller than 8, so multiplication by x can produce at most degree 8; reduction is done via $x^8 = x^4 + x^3 + x + 1 = 0x1B$

```
1 Byte xtime(Byte b)
2   if (b ∧ 0x80)
3       return (0x1B ⊕ (b << 1));
4   else
5       return (b << 1) ;
```

SubBytes

Is defined as $S(x) = f(x^{-1}) = \mathbf{A} \cdot x^{-1} + \mathbf{b}$:

$$\begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} {x^{-1}}_0 \\ {x^{-1}}_1 \\ {x^{-1}}_2 \\ {x^{-1}}_3 \\ {x^{-1}}_4 \\ {x^{-1}}_5 \\ {x^{-1}}_6 \\ {x^{-1}}_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

The finite field inversion is costly, so often this function is done via a look-up in a precomputed table.

However, we will see later, that then when using secret sharing with more than two shares, computing x^{-1} is unavoidable. Lagrange's theorem $x^p \equiv x \mod p$ helps us here: $x^{p-1} \equiv x^{-1}$, and so $x^{-1} = x^{254}$ in a finite field with 256 elements.

Implementation options for small devices

1. Represent the state as an array of 16 bytes. (On 32-bit architectures, collect four bytes within a word)
2. Interleave the KeyExpansion with the Encryption Rounds (generate round keys on the fly).
3. Implement AddRoundKey, ShiftRows, and MixColumns using Boolean arithmetic, using `xtime`.
4. If no or low side channel resilience is needed, implement SubBytes as a table lookup, otherwise, as exponentiation.

ROADMAP

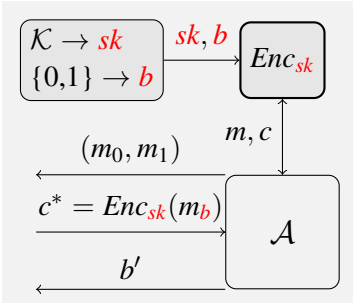
Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

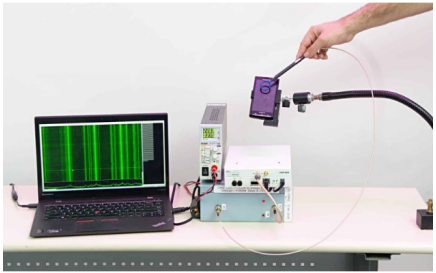
The Power of the Dark Side

Theoretical cryptography: strong, but idealised assumptions about what an adversary gets in real life.

Implementing cryptography: dealing with the mess (leakage through power, EM, execution time, cache access, sound, noise, faults, ...).



(a) Indistinguishable Encryptions



(b) Key Recovery from Phone

Picture to the right taken from the team of authors from https://eprint.iacr.org/2016/129.pdf

Attacking Implementations of Cryptography

In the previous slide we assumed that the adversary has access to an encryption oracle.

In the real world this could be: a smart phone, a car key, a debit card, a credit card, an HSM in a cash machine, a web server, Alexa, Siri, etc.

The access to such oracles can be

- physical: via local access, typically for embedded systems
- logical: via an interface, either local or remote, the latter would be for cloud based systems

The access could be for a prolonged period of time, or short.

Sometimes adversaries can play with duplicates of the system: e.g. embedded devices, or via another account on the cloud service.

The Triad: Timing, Power and EM

With physical access one can do high precision timing, power and EM measurements.

With logical access one can also record I/O data, including network traffic.

Within these channels, there is often a surprising amount of data dependency that relates to secret key usage or other secret data.

Attacks exploiting information from side channels have turned out to be extremely powerful in practice, and they concern **all** types of devices/systems.

We consider some in the context of embedded systems, with the aim of understanding how secret sharing plays a role in securing devices.

Key Recovery via Side Channels

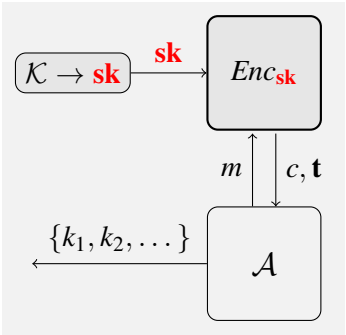


Figure: Key Recover with side channels: KR-LCPA

Most side channel attacks have key recovery as a goal: divide and conquer independent chunks k_i of the secret key.

The adversary supplies some input (m) in this illustration, and they may receive output (c in this example) and a side channel "trace" $\mathbf{t}$.

Univariate side channels (i.e. there is a single measurement, like overall execution time)

Multivariate side channels (i.e. instantaneous power/EM).

Key Recovery via Side Channels

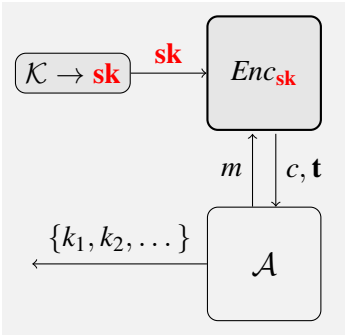


Figure: Key Recover with side channels: KR-LCPA

Notice: the secret key and trace variables are typeset in bold font to indicate that they are vectors/multi-variate variables.

The adversary recovers the (independent) chunks of the key.

How do we define the win condition?

Success Rate: percentage of key recovery attacks which "rank" the correct subkey as most likely.
Key Rank: some (statistical) characteristic of what "rank" the correct key takes among all keys.

Over to You

Suppose that an attack on an AES implementation that recovers the secret key byte-by-byte is considered successfull if the correct (sub) key is within the top 8 key candidates (i.e. the attack has subkey rank 8). This means that for each key byte there are 8 very likely values.

How big is the effort for an adversary to determine the full secret key? Do you think that such an attack is practically feasible?

ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

TIMING

Many optimisation techniques are based on exploiting data dependencies, which reveal information via side channels.

For instance: in the `xtime` function before, we have a data dependent branch condition.

For instance: in some modern embedded processors, **multiplication can terminate early** if one of the multipliers is detected to be 0. This enables side channel attacks.

For instance: in modern processors with (shared) cache, processes can selectively **evict the cache**. The (re)loading time of variables depends if or not they are in the cache, and this enables side channel attacks.

TIMING (FIXING XTIME)

```
1 Byte xtime(Byte b)
2   if (b ∧ 0x80)
3       return (0x1B ⊕ (b << 1));
4   else
5       return (b << 1) ;
```

The `xtime` function takes a byte b (which represents an element of $\mathbb{Z}_{2^8}$) and performs a (polynomial) multiplication by x.

The condition in the `if` statement is data dependent: b represents a state byte. Hence it depends on the message, and because MixColumns happens after AddRoundKey, it also depends (at least) one byte of the round key.

Therefore, this implementation provides a timing side channel, and a key recovery attack will work.

TIMING (FIXING XTIME)

```
1 Byte xtime(Byte b)
2   if (b ∧ 0x80)
3       return (0x1B ⊕ (b << 1));
4   else
5       return (b << 1) ;
```

```
1 Byte xtime(Byte b)
2   Byte t = (b << 1);
3   b = b >> 7
4   b = b · 0x1B;
5   b = b ⊕ t;
6   return b
```

To fix `xtime` we need to rewrite the function so it performs the check for the most significant bit always:

- multiply (via a left shift) and store the result in a new variable t
- shift the MSB down into the LSB and multiply with the reduction factor
- if the LSB was 1 then $b = 0x1B$, otherwise $b = 0$
- we XOR t with b, which performs the polynomial reduction iff the MSB was 1

Over to You

In the previous slide I claim that this is a fix to the timing leakage in `xtime`. Which is somewhat true, but it depends on a condition that I actually discussed before. What might be the remaining caveat (i.e. source of timing leakage) in the "fix"?

Data dependent power and EM illustrated on inverter

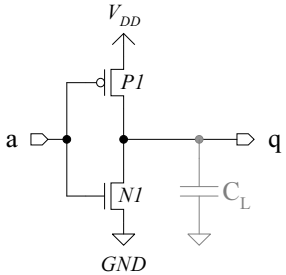


Figure: CMOS inverter.

The inverter is based on two complementary networks of physical elements.

Each network has one transistor (top network: $P1$, bottom network: $N1$).

The transistors conduct in complementary ways: if $P1$ "opens", $N1$ "closes".

If the inverter $P1$ is conducting and $N1$ is insulating if the input a is set to GND.

Vice versa, $P1$ is insulating and $N1$ is conducting if the input a is set to V_{DD}.

Data dependent power and EM illustrated on inverter

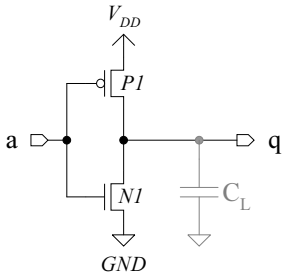


Figure: CMOS inverter.

Transition	Type of power consumption
$0 \to 0$	static
$0 \to 1$	static + dynamic
$1 \to 0$	static + dynamic
$1 \to 1$	static

Table: Output transitions of the CMOS inverter and the corresponding power consumption.

There is more current moving through the circuit when values change. This implies that the power consumption depends on the data. The exact dependency depends on the physical properties of the transistors and their connection wires: the current has to "overcome" their resistance, which is captured by C_L in the picture.

The current flow changes the electromagnetic field, which implies that it is also data dependent.

ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

Single vs Differential Input Attacks

Classical security notions distinguish between “passive” adversaries depending on how an adversary can interact with the encryption oracle: “passive” means that an adversary gets only one challenge ciphertext, whereas CPA implied that the adversary can send additional messages to an encryption oracle.

A similar distinction is made in the context of side channel attacks. They are in principle also “passive”, because adversaries don’t “interfere” with the encryption oracle. However, if an adversary just gets a single side channel trace, the we call this **single trace attack** (or simple side channel analysis). If an adversary gets traces for multiple encryptions then we talk about **differential input attacks** (or differential side channel analysis).

There are protocols, where a key would only be used once (ephemeral keys e.g. in the context of signatures), and there are protocols where the same secret key is used for bulk encryption of data.

Single Trace Attacks on AES

```
1  AES_Encrypt(ByteArray m, ByteArray k)
2   ByteArracy c
3   state = m
4   rk = KeyExpansion(k)
5   AddRoundKey(state, rk_0)
6   for r = 1 step 1 to 9
7               SubBytes(state)
8               ShiftRows(state)
9               MixColumns(state)
10              AddRoundKey(state, rk_r)
11   end
12   SubBytes(state)
13   ShiftRows(state)
14   AddRoundKey(state, rk_10)
15  c = state;
```

SINGLE TRACE: AES

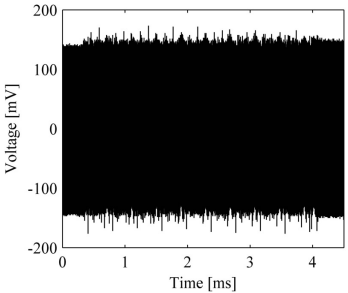


Figure: Full AES encryption.

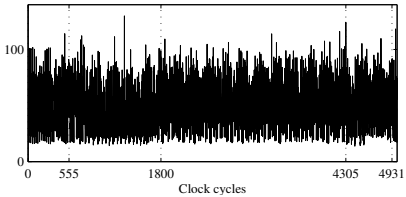


Figure: One round of AES

In case of an algorithm like AES, there is no simple association between “flow” and secret key bits. Therefore we can gather useful information by looking/analysing a single trace, but unless some a priori information is available, we cannot extract the key as such.

Differential Input Attacks

In this module we will use power traces to illustrate all attacks. In the literature, and on many websites, differential input attacks using power traces are referred to as “Differential Power Attacks” (DPA).

In the context of attacks using EM traces, some people then use DEMA to refer to differential input attacks, and in the context of attacks using timing information, the abbreviation DTA might be used.

Some people/website mistakenly link the term differential, to the use of the “distance of means test” during the statistical analysis.

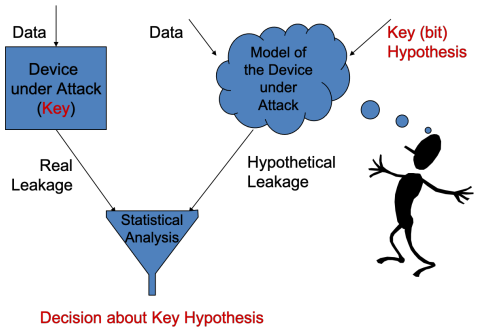
I use the term “DPA style” or “differential style” for any attack that uses differential inputs.

Differential Attacks

DPA style or differential attacks work whenever we have a static secret that is used by crypto function with (at least partially) known inputs or outputs.

Depending on the type of scheme, the device and the implementation strategy, one can often make guesses about just a few bits of the key at a time: i.e. we use a **divide and conquer** strategy.

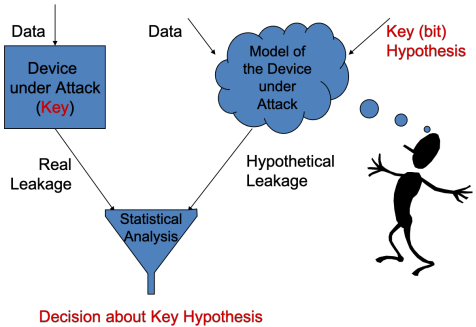
Because of the statistical analysis, it is possible to exploit even the “tiniest” of leakage.



Differential Attacks

There are a number of choices that are necessary of "configuring" a concrete attack:

- The model: intermediate value and leakage model
- The measurements: what side channel and how/what to measure
- The statistical analysis



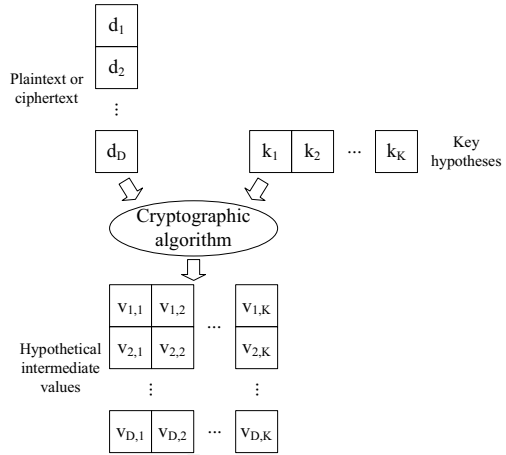
The picture implies five distinctive steps/choices, which we now run through.

Differential Attacks: Intermediate Value

1. Choose an intermediate variable (aka target variable) that depends on only a small part of the input and the unknown key.

AES: single state byte after AddRoundKey, SubBytes, ShiftRows, or, two or more state bytes within/after MixColumns; first/last round.

3. Computing all combinations of key guesses and inputs, using the crypto algorithm's steps, then gives predicted/hypothetical intermediate values.

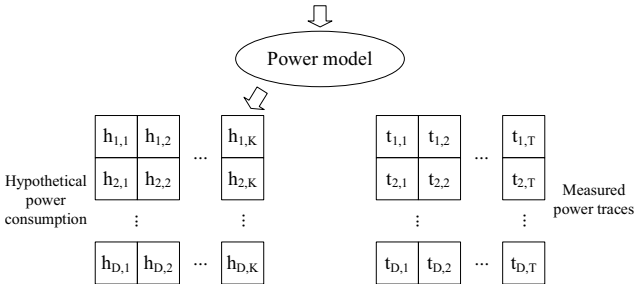


Differential Attacks: Record Traces

The choice of intermediate value will probably guide how/what to measure.

If plaintexts are available, and a value in the first round is selected, record a time period at the start of encryption.

2. For each input, we record a trace.

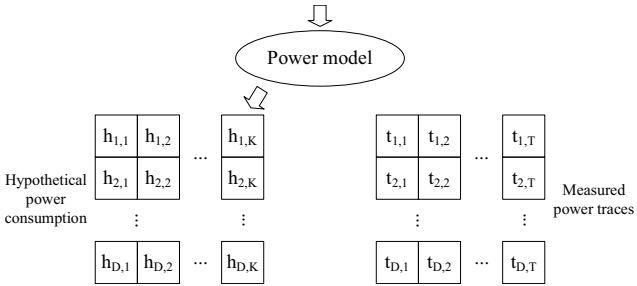


Differential Attacks: Hypothetical Leakages

We already have, for each input, all possible hypothetical intermediate values.

We now need to decide how to "interpret" these for the purpose of statistical inference: how do the predicted values relate to the actual observations?

4. We use an adversarial leakage model (in the case of power analysis, we call it a power model) for this purpose.

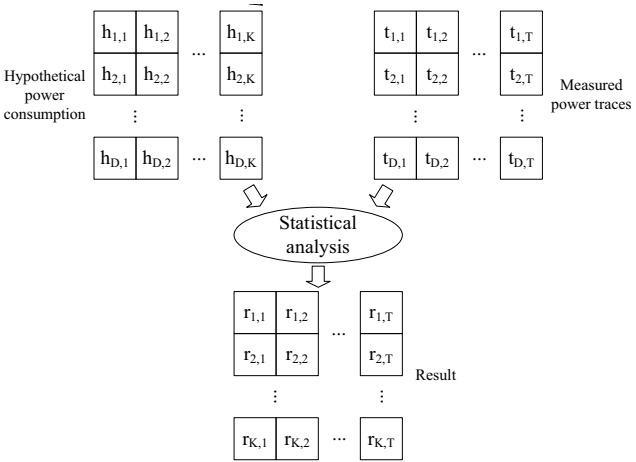


Differential Attacks: Hypothetical Leakages

For each input, we have hypothetical leakage values and a side channel trace.

We know wish to determine which of the “columns” in our hypothetical values indicates the value of the correct key.

5. We do this by using statistics: each column is “compared” with all trace points by virtue of a distinguisher.



ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

THREE PRONGED APPROACH

Protocol level: change key often

Hiding: remove link between values stored in the device and leakage; can be done by modifying hardware or software.

Masking: remove link between predictable intermediate values and values that are actually held in the device; can be done in hardware or software.

THREE PRONGED APPROACH

In practice, all three approaches are used: sessions keys are seen as good practice (so only one trace per key), hiding makes straightforward DPA attacks more trace hungry, masking makes straightforward DPA attacks impossible.

More scientific: the effort for an adversary is measured in the number of traces that the require. Hiding countermeasures increase this effort linearly, whereas masking (aka secret sharing) potentially increases the effort exponentially in the number of shares.

ROADMAP

Context: secure implementations
- Advanced Encryption Standard
- Implementing AES

Side Channel Attacks
- General principle of Key Recovery via SCA
- Sources of Timing, Power, and EM Leakage
- Single Input and Differential Input Attacks
- Countermeasures
- DPA on an Unprotected AES Implementation

AES IMPLEMENTATION

We target an implementation of AES on a typical 8-bit microprocessor. This means that the data-bus can transfer 8 bits at a time, and all operations like `XOR`, `AND`, ..., that the ALU implements work on 8 bit values.

We target the SubBytes output of AES in round 1.

We know that the implementation for SubBytes is based on a stored table: i.e. the SubBytes output will be transferred over the data-bus. Data-bus leakage is often strongly related to the Hamming weight of the data.

We configure our equipment so that it records just the beginning of the encryption operation. We store 1000 traces as variable/matrix T, and the plaintext input.

AES IMPLEMENTATION, CONT.

We only need to use the first byte of the 16 byte plaintext to guess the output of the first SubBytes. Using our 1000 inputs, and a single byte key guess, we calculate the values $v_{i,j} = S(d_i \oplus k_j)$, where $d_1, \ldots, d_{1\,000}$ are the first byte of each of the 1 000 plaintexts and $k_j = j - 1$ with $j = 1, \ldots, 256$.

The matrix $\mathbf{V}$ has hence a size of $1\,000 \times 256$ values (step 3).

Next we map this matrix to hypothetical power values using the Hamming weight leakage model, leading to a matrix $\mathbf{H}$, $h_{i,j} = HW(v_{i,j})$.

We use the correlation coefficient to determine the linear relationship between the columns $\mathbf{h}_i$ and $\mathbf{t}_j$ for $i = 1, \ldots, K$ and $j = 1, \ldots, T$.

AES IMPLEMENTATION, CONT.

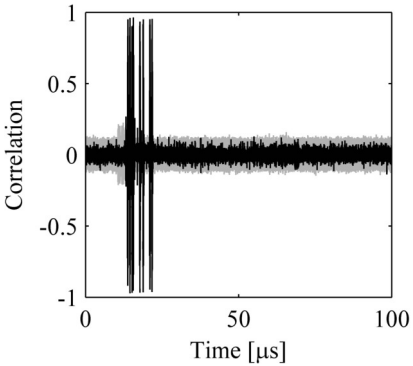


Figure: Correct key hypothesis 225 is plotted in black, while all other key hypotheses are plotted in grey.

The result gives the column 225 the highest correlation values. This indicates that the first byte of the secret key has the value 225.

AES IMPLEMENTATION, CONT.

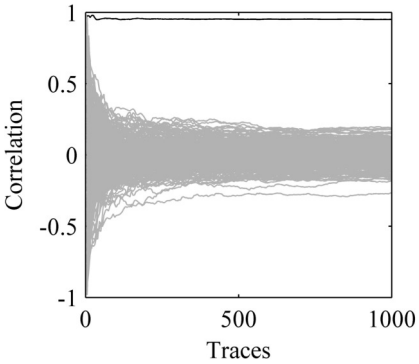


Figure: Correct key hypothesis 225 is plotted in black, while all other key hypotheses are plotted in grey.

This plot indicates that by far fewer traces would suffice for identifying this first byte.

Over to You

Consult the resp. lab sheet to get going with DPA style attacks. Code and data is available on the module website.